

DIGITAL LOGIC DESIGN

(CE_118)

CHAPTER 4:

PROCESSOR DESIGN

COMPUTER ENGINEERING

(part_2)

Outline

➤ We introduced new concepts:

- ✓ Instruction sets
- ✓ Instruction types
- ✓ Addressing modes
- ✓ Instruction-execution cycle
- ✓ Processor design flow

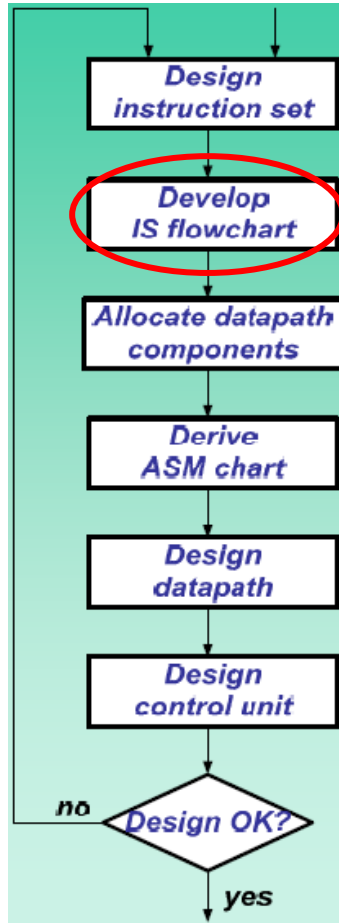
➤ Including

- ✓ instruction set design,
- ✓ instruction set flowcharts,
- ✓ component allocation,
- ✓ ASM charts
- ✓ processor architecture

➤ We have demonstrated processor design

- ✓ 16-bit CISC design
- ✓ 32-bit RISC design
 - data-forwarding
 - branch prediction

CISC Design: Instruction-set flowchart



Design process

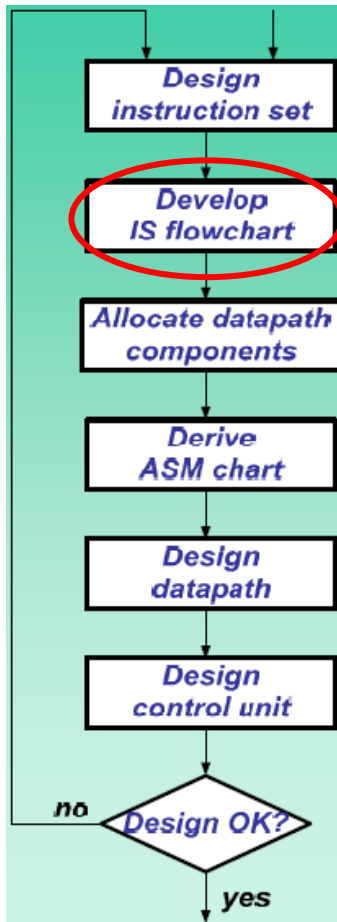
Instruction-set flowchart

- Does not presume any architectural details
- Does not any particular processor datapath
- Does not consider any timing constraints or clock cycle duration

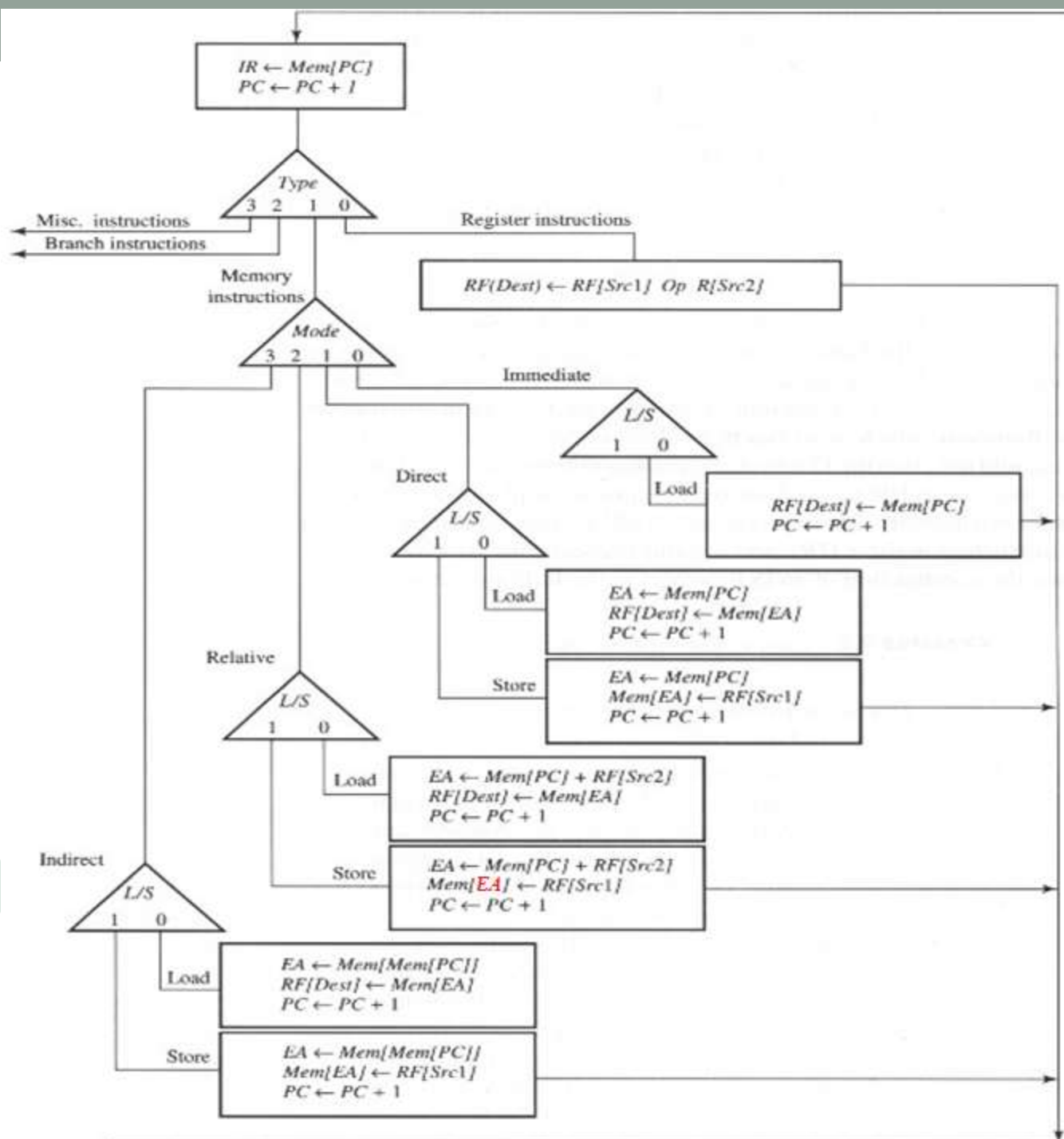
Purpose:

- Give the order in which the operations specified by each instruction will be executed.

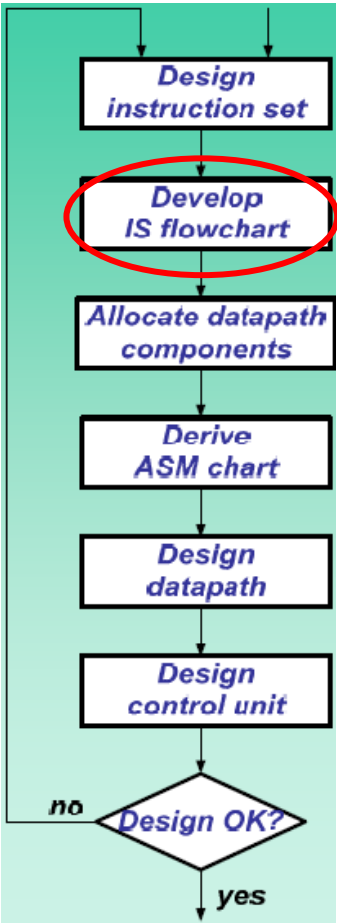
CISC Design: Instruction-set flowchart



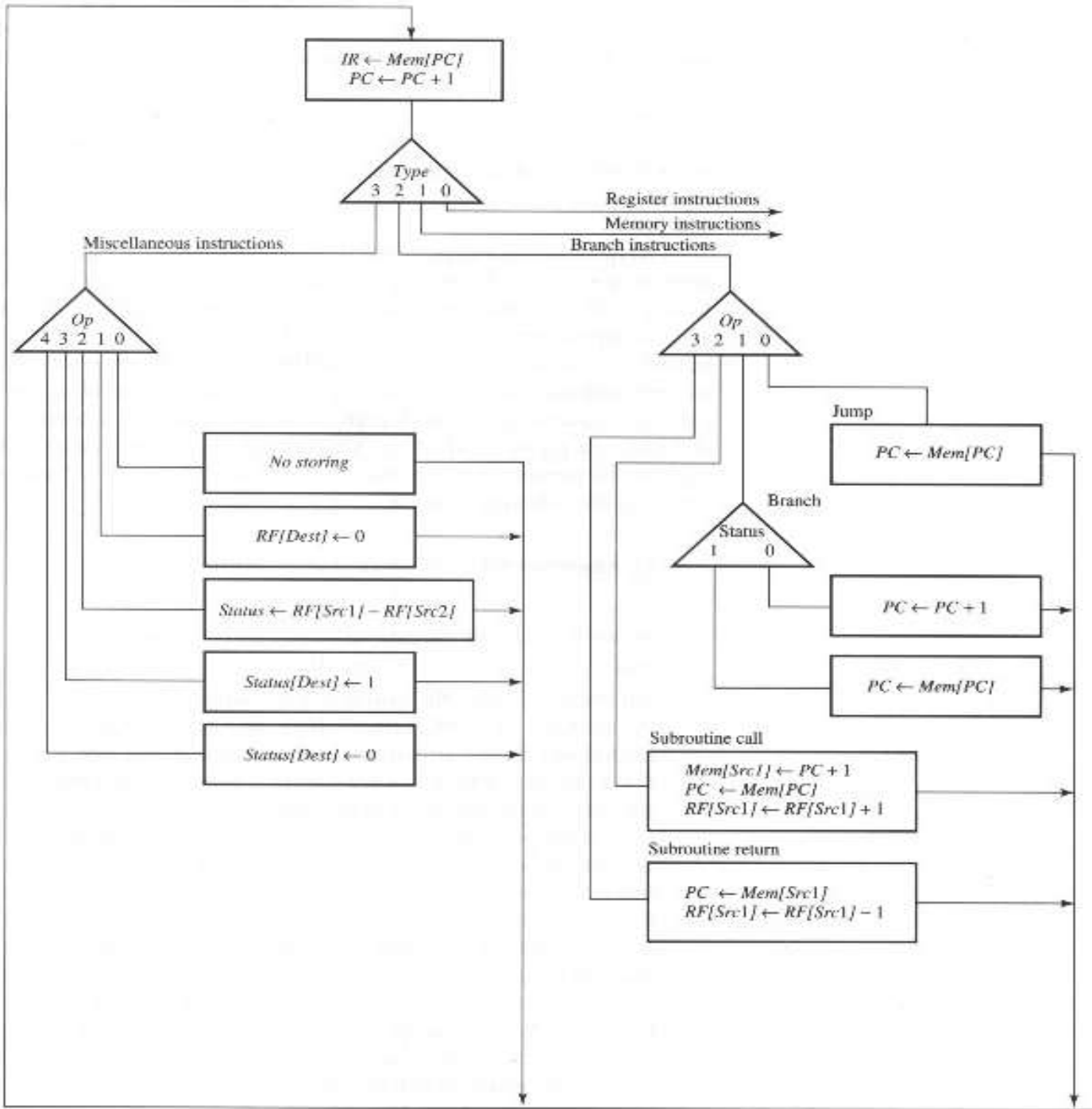
Design process



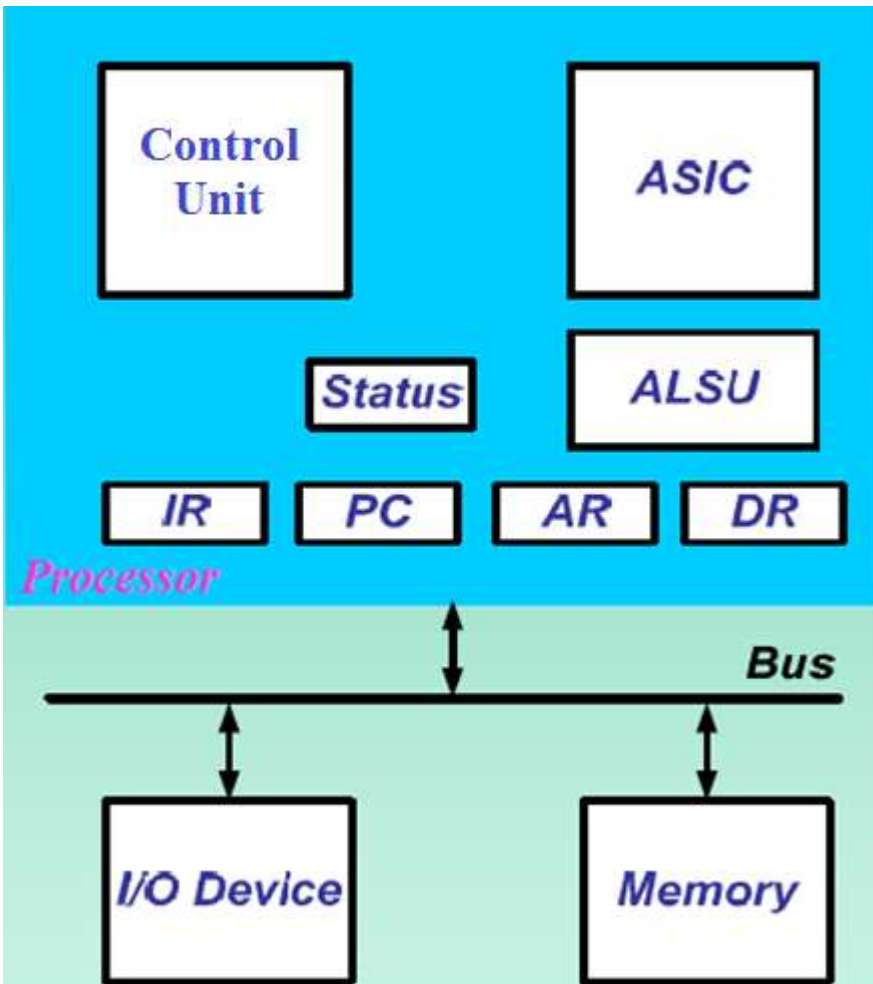
CISC Design: Instruction-set flowchart (cont.)



Design process



Component allocation for the 16-bit processor



Components: 64K x 16 Memory

8 x 16 Register file

ALSU (S: shift)

Instruction register (IR)

Program counter (PC)

Address register (AR)

Data register (DR)

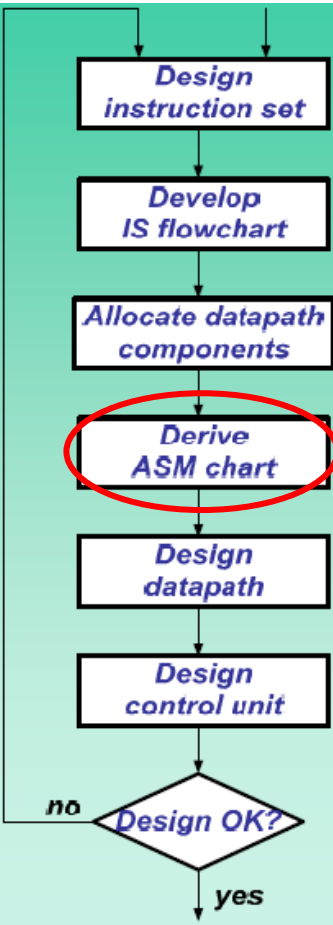
Status register (Status)

Control unit

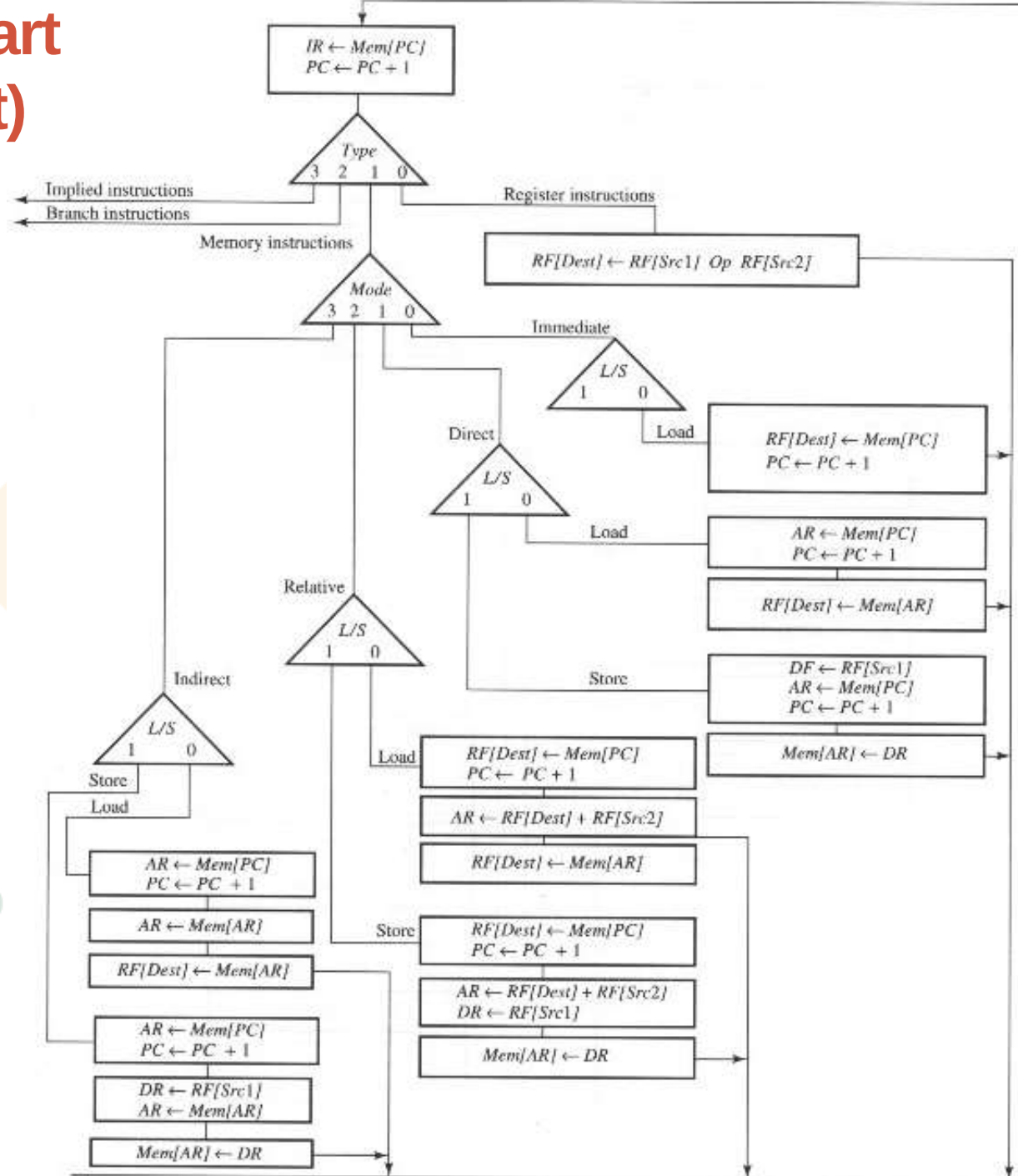
ASIC

AR, DR: needed to shorten clock period and improve the performance of the processor

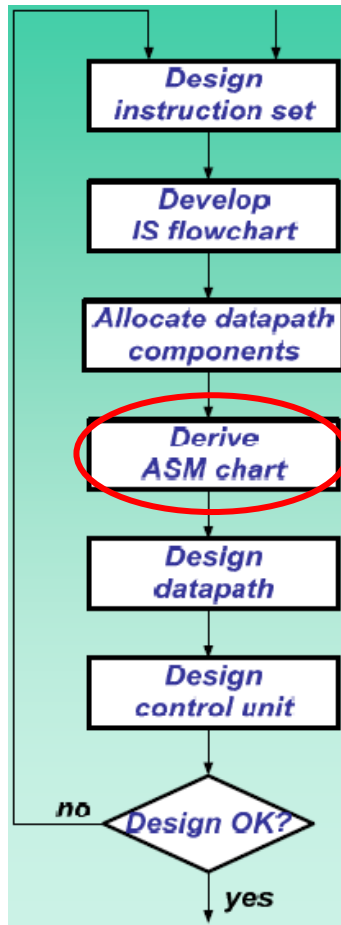
Processor ASM chart (scheduled IS chart)



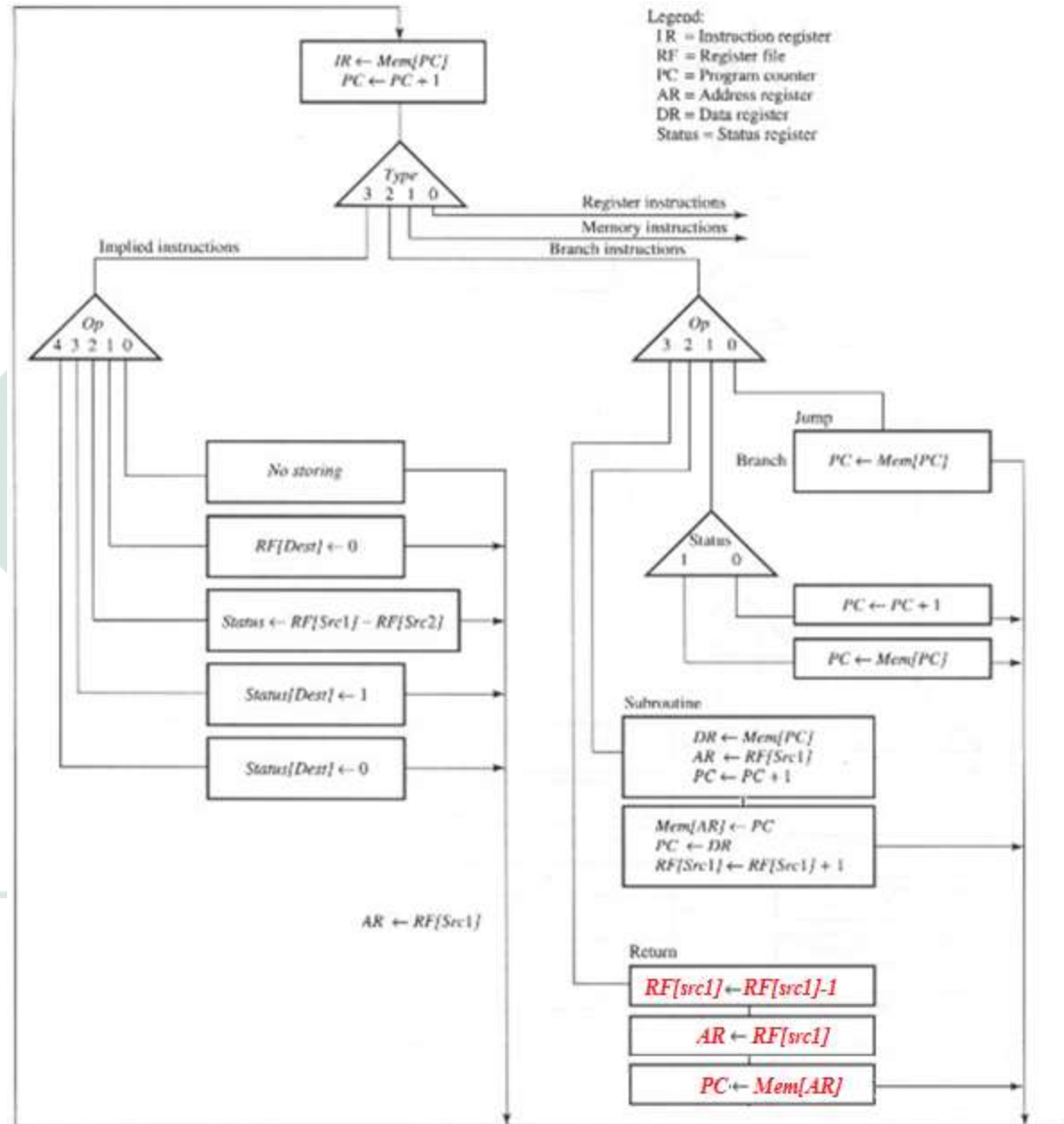
Design process



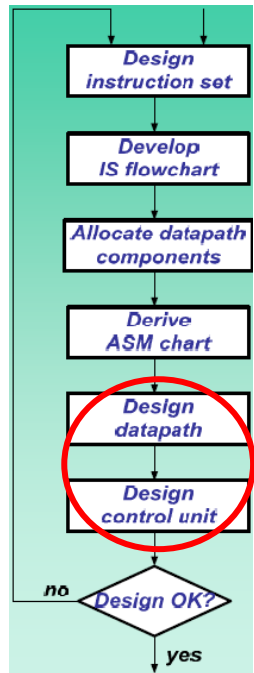
Processor ASM chart (scheduled IS chart)



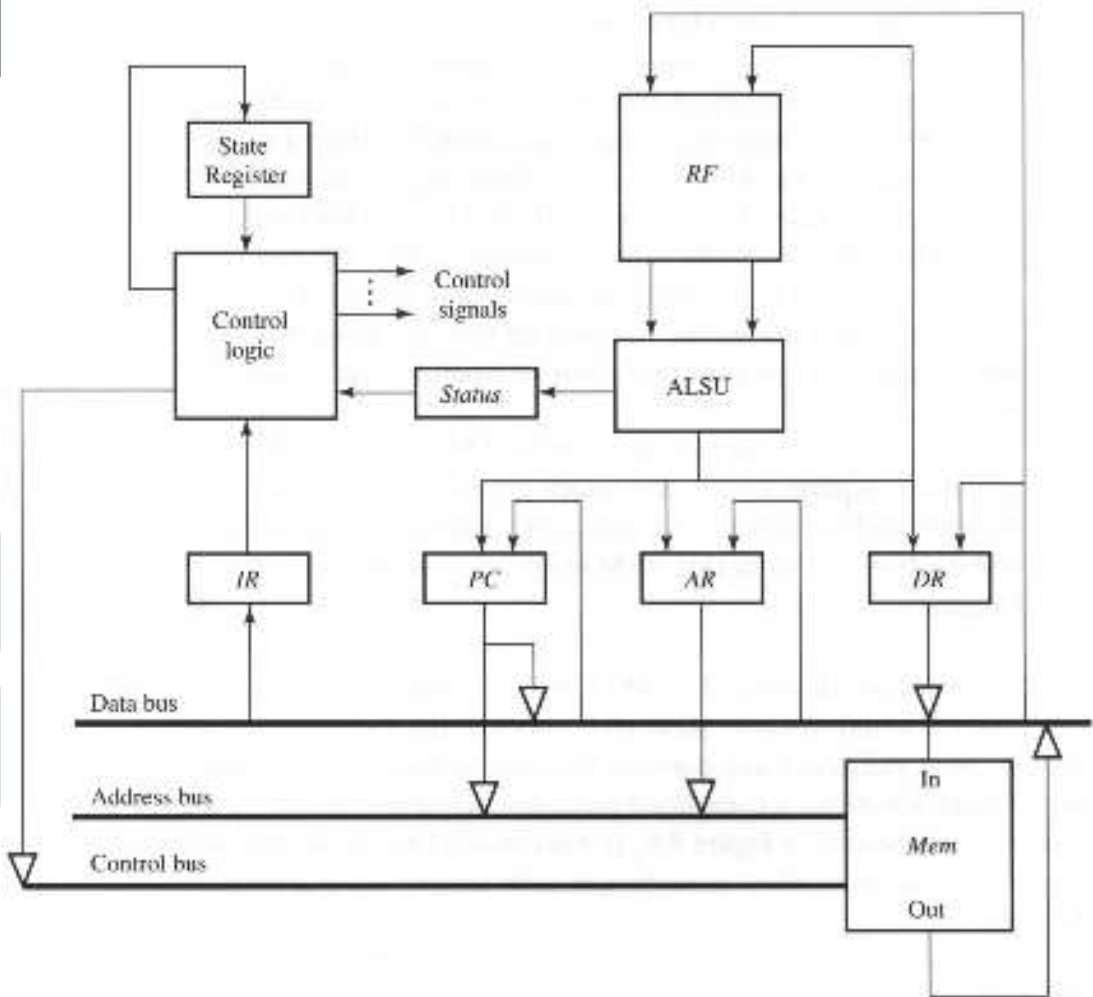
Design process



Processor schematic



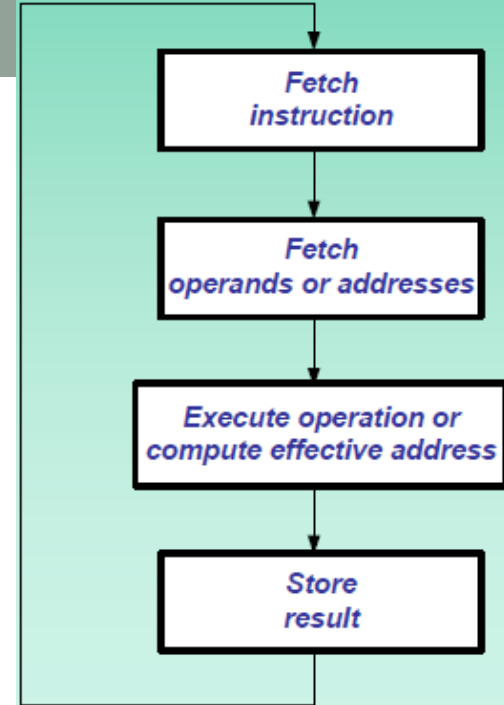
Design process



- Schematic is obtained by connecting the processor components according to the ASM chart, that is, by adding a connection whenever data or an instruction is moved from one component to another.
- Components with several connections at the same input port require selectors for that particular input port.

Reduced instruction-set cycle (RISC design)

- Register and misc. instructions do not need address fetch and effective address computation
- Memory and control instructions do not need operand fetch and operation execution
- Share operand fetch and address fetch
- Share operand execution and effective address computation
- Thus, instruction cycle reduced to 4 steps



Instruction-execution cycle

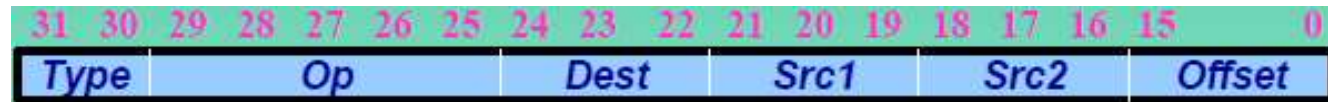
Clock cycle	0	1	2	3	4	5
Fetch instruction	I_1	I_2	I_3	I_4	I_5	I_6
Fetch operands or addresses		I_1	I_2	I_3	I_4	I_5
Execute operation or compute effective address			I_1	I_2	I_3	I_4
Store result				I_1	I_2	I_3

Pipelined execution

Reduced instruction-set for a 32-bit processor (RISC type)

a) Register Instructions

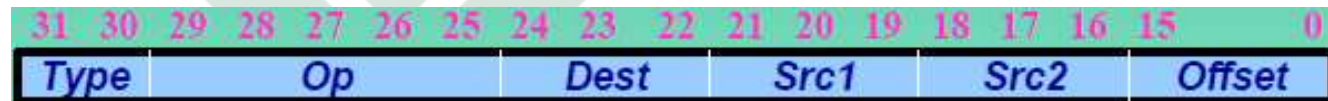
arithmetic, logic, move
and shift



<u>Name</u>	<u>Action</u>
Op Dest, Src1, Src2	$RF(Dest) \leftarrow RF[Src1] \text{ Op } RF[Src2]$
Op Dest, Src1, Src2	$RF(Dest) \leftarrow RF[Src1] \text{ Op Constant}$
Move Dest, Src1	$RF(Dest) \leftarrow RF[Src1]$
Shift Dest, Src1, Constant	$RF(Dest) \leftarrow RF[Src1] \text{ shift Constant}$

b) Memory instructions

load and store



<u>Name</u>	<u>Action</u>
L immU Dest	$RF[Dest(31...16)] \leftarrow Offset$
L immL Dest	$RF[Dest(15...0)] \leftarrow Offset$
L rel Dest, Src2, Offset	$RF[Dest] \leftarrow Mem[RF[Src2] + Offset]$
S rel Src1, Src2, Offset	$Mem[RF[Src2] + Offset] \leftarrow RF[Src1]$

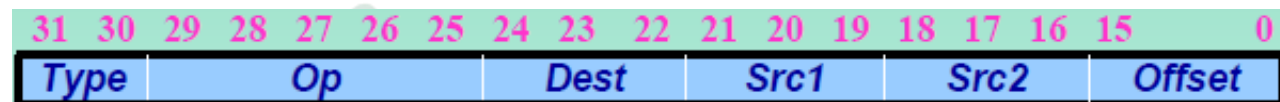
2 addressing moed:

- Immediate
- Relative

Reduced instruction-set for a 32-bit processor (RISC type)

c) Control instructions

jump and branch



Beq (=)

Bgre (>)

Bgoeq ($\geq$)

Bless (<)

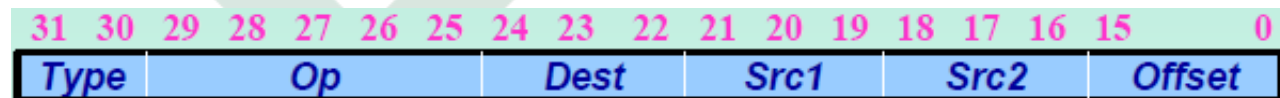
Bloeq ($\leq$)

Bneq ($\neq$)

<u>Name</u>	<u>Action</u>
Jump Offset	$PC \leftarrow PC + \text{offset}$
Jump Src2, Offset	$PC \leftarrow RF[\text{Src2}] + \text{offset}$
Brel Src1, Src2, Offset	$PC \leftarrow PC+1$ if $RF[\text{Src1}]$ not rel $RF[\text{Src2}]$ $PC \leftarrow PC+\text{Offset}$ if $RF[\text{Src1}]$ rel $RF[\text{Src2}]$

d) Miscellaneous instructions

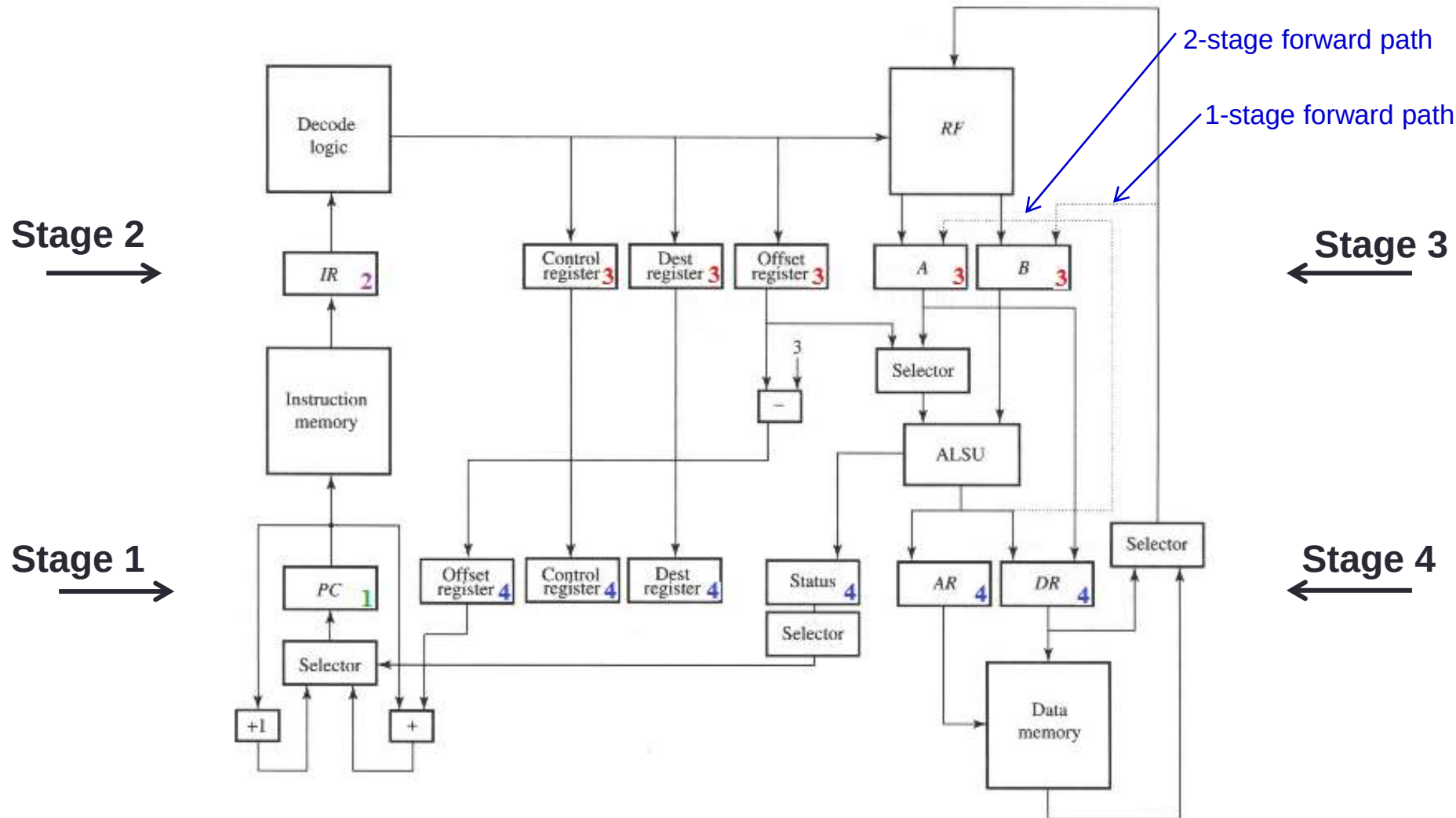
no-op, clear,
set and reset



<u>Name</u>	<u>Action</u>
No-op	Do nothing
Clear Dest	$RF[\text{Dest}] \leftarrow 0$
Sstat Dest	$\text{status}[\text{Dest}] \leftarrow 1$
Rstat Dest	$\text{status}[\text{Dest}] \leftarrow 0$

RISC block diagram

- Four-stage pipeline
- Separate instruction and data memories
- Add control register in each pipeline stage
- Pipeline stalling (flushing) for control instructions



RISC operation for a 3-line program

$x = a + b$

$y = b - c$

$z = c + d$

Source program

Address	Instruction
100	Load a, base, off
101	Load b, base, off
102	Load c, base, off
103	Load d, base, off
104	Add x, a, b
105	Sub y, b, c
106	Add z, c, d
107	Store x, base, off
108	Store y, base, off
109	Store z, base, off
110	—

Assembly program

Concurrently:

$n+3$ clock cycles to execute n instructions

Sequentially:

$4n$ clock cycles for n instructions (no pipeline)

Clock cycle	0	1	2	3	4	5	6	7	8	9	10	11	12	13
Read PC	100	101	102	103	104	105	106	107	108	109	110			
Write IR	Load	Load	Load	Load	Add	Sub	Add	Store	Store	Store				
Write A						a	b	c	x	y	z			
Write B		base	base	base	base	b	c	d	base	base	base			
Write AR			base +off	base +off	base +off	base +off				base +off	base +off	base +off		
Write DR							a+b	b-c	c+d	x	y	z		
Write RF				a	b	c	d	x	y	z				
Write Mem											x	y	z	
Write PC	101	102	103	104	105	106	107	108	109	110	111	112	113	114

Timing diagram

RISC operation for a 2-line program with data dependencies

sum = a + b

total = sum + c

Source program

Data dependence:

5 (or 45%) of 11 instructions are No-op instructions → decrease substantially performance of pipelined processor

Address	Instruction
100	Load a, base, offa
101	Load b, base, offb
102	Load c, base, offc
103	No-op
104	Add sum, a, b
105	No-op
106	No-op
107	Add total, c, sum
108	No-op
109	No-op
110	Store total, base, offt

Assembly program

Clock cycle	0	1	2	3	4	5	6	7	8	9	10	11	12	13
Read PC	100	101	102	103	104	105	106	107	108	109	110			
Write IR	Load	Load	Load	No-op	Add	No-op	No-op	Add	No-op	No-op	Store			
Write A						a			Sum			total		
Write B		base	base	base		b			c			base		
Write AR			base +offa	base +offb	base +offc								base +offt	
Write DR							a+b			sum+c			total	
Write RF				a	b	c		sum			total			
Write Mem														total
Write PC	101	102	103	104	105	106	107	108	109	110	111	112	113	114

Timing diagram

RISC operation for a 2-line program with data-forwarding

sum = a + b
total = sum + c

Source program

Address	Instruction
100	Load a, base, off
101	Load b, base, off
102	Load c, base, off
103	Add sum, a, b
104	Add total, c, sum
105	Store total, base, off

Assembly program

Clock cycle	0	1	2	3	4	5	6	7	8
Read PC	100	101	102	103	104	105			
Write IR	Load	Load	Load	Add	Add	Store			
Write A					a	$F2[a+F1(b)]$	$F2[F2(a+F1(b)) + F1(c)]$		
Write B		base	base	base	$F1(b)$	$F1(c)$	base		
Write AR			base +off	base +off	base +off			base +off	
Write DR						$a + F1(b)$	$F2[a+F1(b)] + F1(c)$	$F2[F2(a+F1(b)) + F1(c)]$	
Write status									
Write RF				a	b	c	sum	total	
Write Mem									total
Write PC	101	102	103	104	105	106	107	108	109

Timing diagram

RISC operation for a 2-line program with data-forwarding

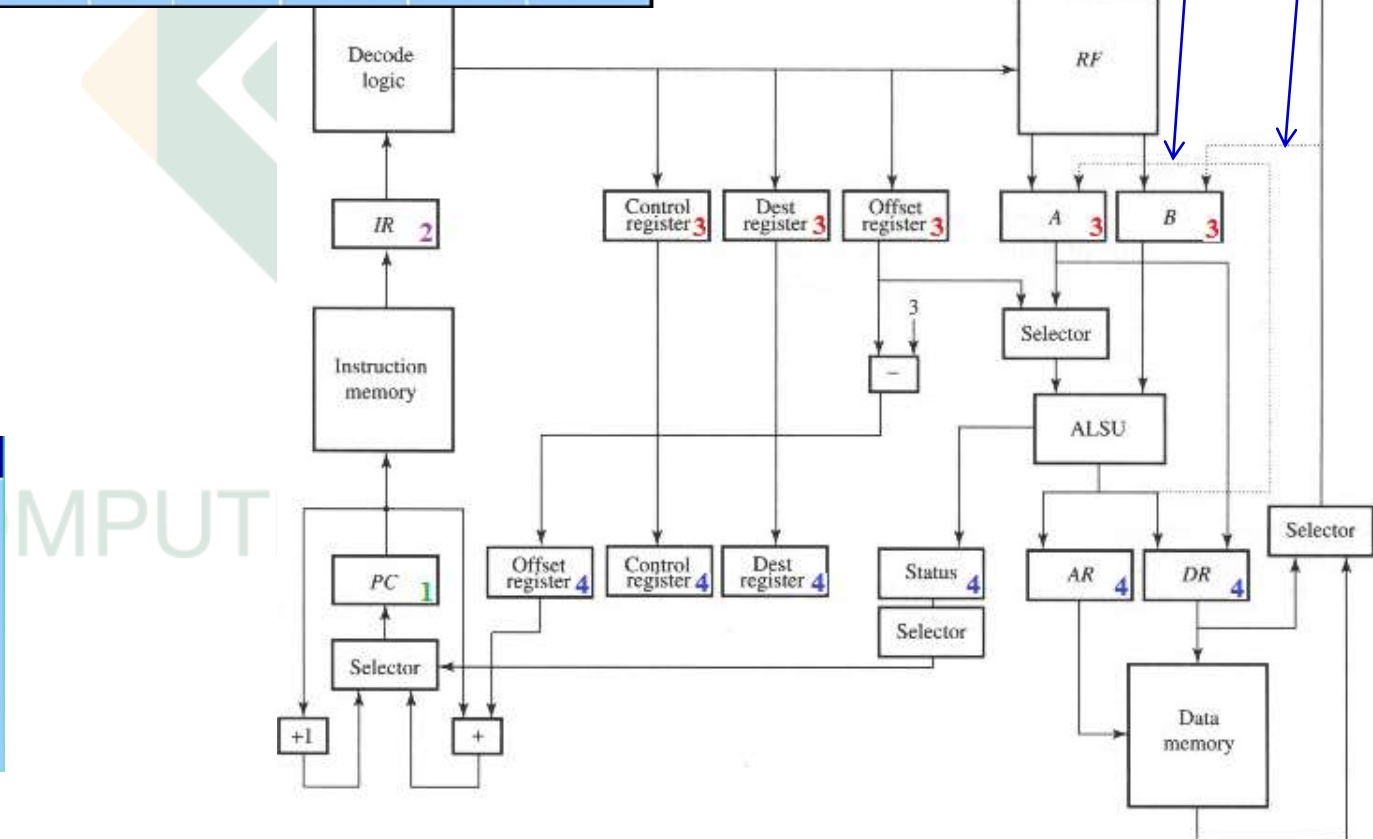
Clock cycle	0	1	2	3	4	5	6	7	8
Read PC	100	101	102	103	104	105			
Write IR	Load	Load	Load	Add	Add	Store			
Write A					a	$F2[a+F1(b)]$	$F2[F2(a+F1(b)) + F1(c)]$		
Write B		base	base	base	$F1(b)$	$F1(c)$	base		
Write AR			base + off	base + off	base + off			base + off	
Write DR						$a + F1(b)$	$F2[a+F1(b)] + F1(c)$	$F2[F2(a+F1(b)) + F1(c)]$	
Write status									
Write RF				a	b	c	sum	total	
Write Mem									total
Write PC	101	102	103	104	105	106	107	108	109

2-stage
forward path

1-stage
forward path

sum = a + b
total = sum + c

Address	Instruction
100	Load a, base, off
101	Load b, base, off
102	Load c, base, off
103	Add sum, a, b
104	Add total, c, sum
105	Store total, base, off



RISC operation without branch prediction

```

If a ≥ b then
  begin
    max = a
    min = b
  end
else
  begin
    max = b
    min = a
  end
endif

```

Source program

Address	Instruction
100	Bgoeq a, b, +10
101	No-op
102	No-op
103	No-op
104	Move max, b
105	Move min, a
106	Jump + 6
107	No-op
108	No-op
109	No-op
110	Move max, a
111	Move min, b
112	—

Assembly program

Clock cycle	0	1	2	3	4	5	6	7	8	9	10
Read PC	100	101	102	103	104	105	106	107	108	109	112
Write IR	Bgoeq	No-op	No-op	No-op	Move	Move	Jump	No-op	No-op	No-op	
Write A		a				b	a				
Write B		b									
Write AR											
Write DR							b	a			
Write status			a ≤ b								
Write RF								max	min		
Write Mem											
Write PC	101	102	103	104	105	106	107	108	109	112	

Timing diagram when branch is not taken

Clock cycle	0	1	2	3	4	5	6	7	8	9	10
Read PC	100	101	102	103	110	111	112				
Write IR	Bgoeq	No-op	No-op	No-op	Move	Move					
Write A		a				a	b				
Write B		b									
Write AR											
Write DR							a	b			
Write status			a ≥ b								
Write RF								max	min		
Write Mem											
Write PC	101	102	103	110	111	112					

Timing diagram when branch is taken

RISC operation with branch prediction

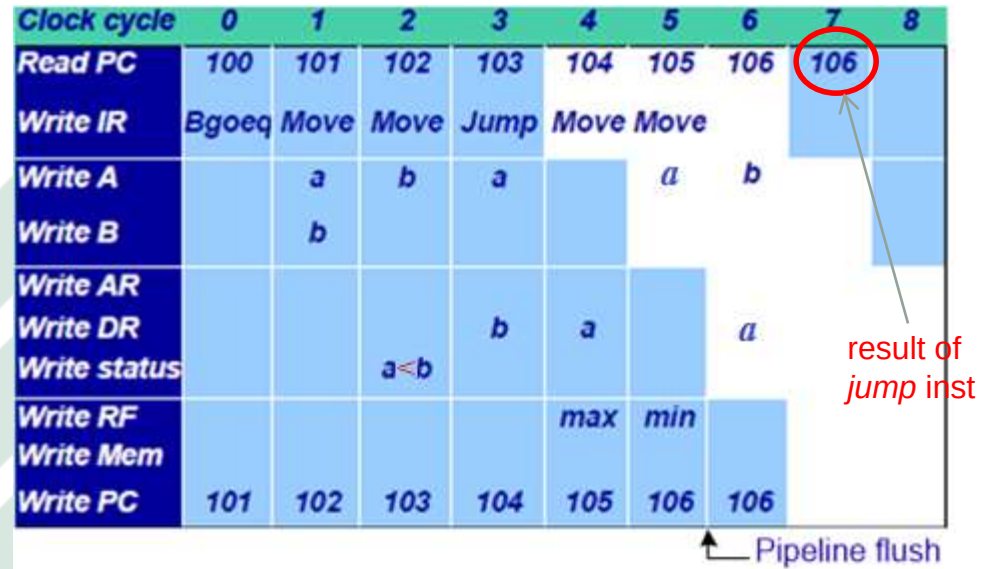
```

If a ≥ b then
  begin
    max = a
    min = b
  end
else
  begin
    max = b
    min = a
  end
endif
  
```

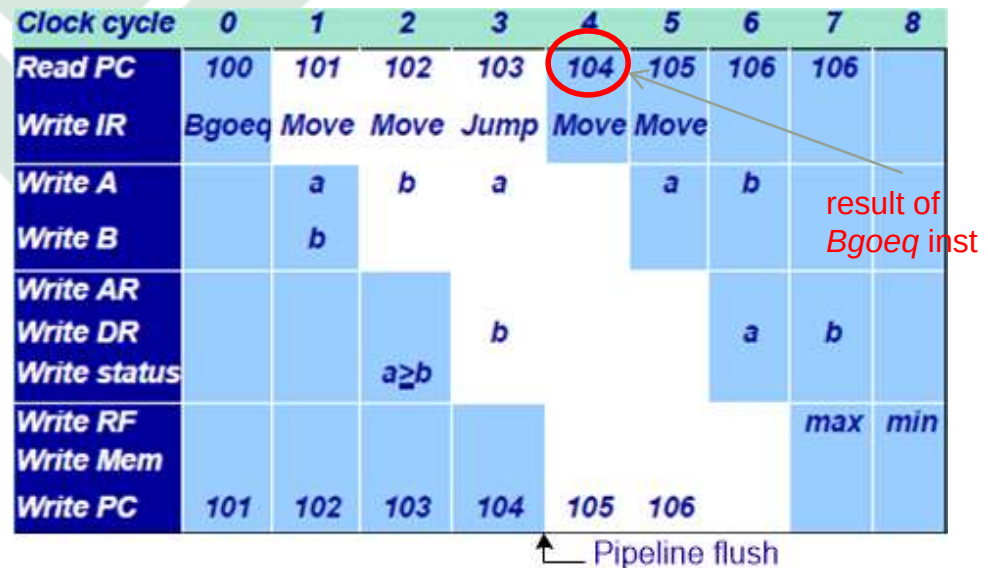
Source program

Address	Instruction
100	Bgoeq a, b, +4
101	Move max, b
102	Move min, a
103	Jump + 3
104	Move max, a
105	Move min, b
106	—

Assembly program



Timing diagram when branch is not taken



Timing diagram when branch is taken

Chapter Summary

➤ **We introduced new concepts:**

- ✓ Instruction sets
- ✓ Instruction types
- ✓ Addressing modes
- ✓ Instruction-execution cycle
- ✓ Processor design flow

➤ **Including**

- ✓ instruction set design,
- ✓ instruction set flowcharts,
- ✓ component allocation,
- ✓ ASM charts
- ✓ processor architecture

➤ **We have demonstrated processor design:**

- ✓ 16-bit CISC design
- ✓ 32-bit RISC design
 - data-forwarding
 - branch prediction

Bài tập

➤ Biểu diễn lệnh sau:

✓ $a = b + c;$

✓ $x = y + z$

Viết lệnh assembly

Biểu diễn lược đồ thời gian.

➤ Thêm lệnh

✓ $a = b + z$

Viết lệnh assembly

Biểu diễn lược đồ thời gian

Bài tập

➤ Biểu diễn lệnh sau trong 2 trường hợp:

- Dùng kỹ thuật data forwarding
- Không dùng kỹ thuật data forwarding.

✓ $a = b + c;$

✓ $x = y + z;$

✓ $t = x + z$

Viết lệnh assembly

Biểu diễn lược đồ thời gian.

COMPUTER ENGINEERING

Bài tập

- Biểu diễn lệnh sau trong 2 trường hợp:
 - Dùng kỹ thuật tiên đoán
 - Không dùng kỹ thuật tiên đoán.

```
if(a < b)
    max = a
else
    max = b
```

Viết lệnh assembly

Biểu diễn lược đồ thời gian